

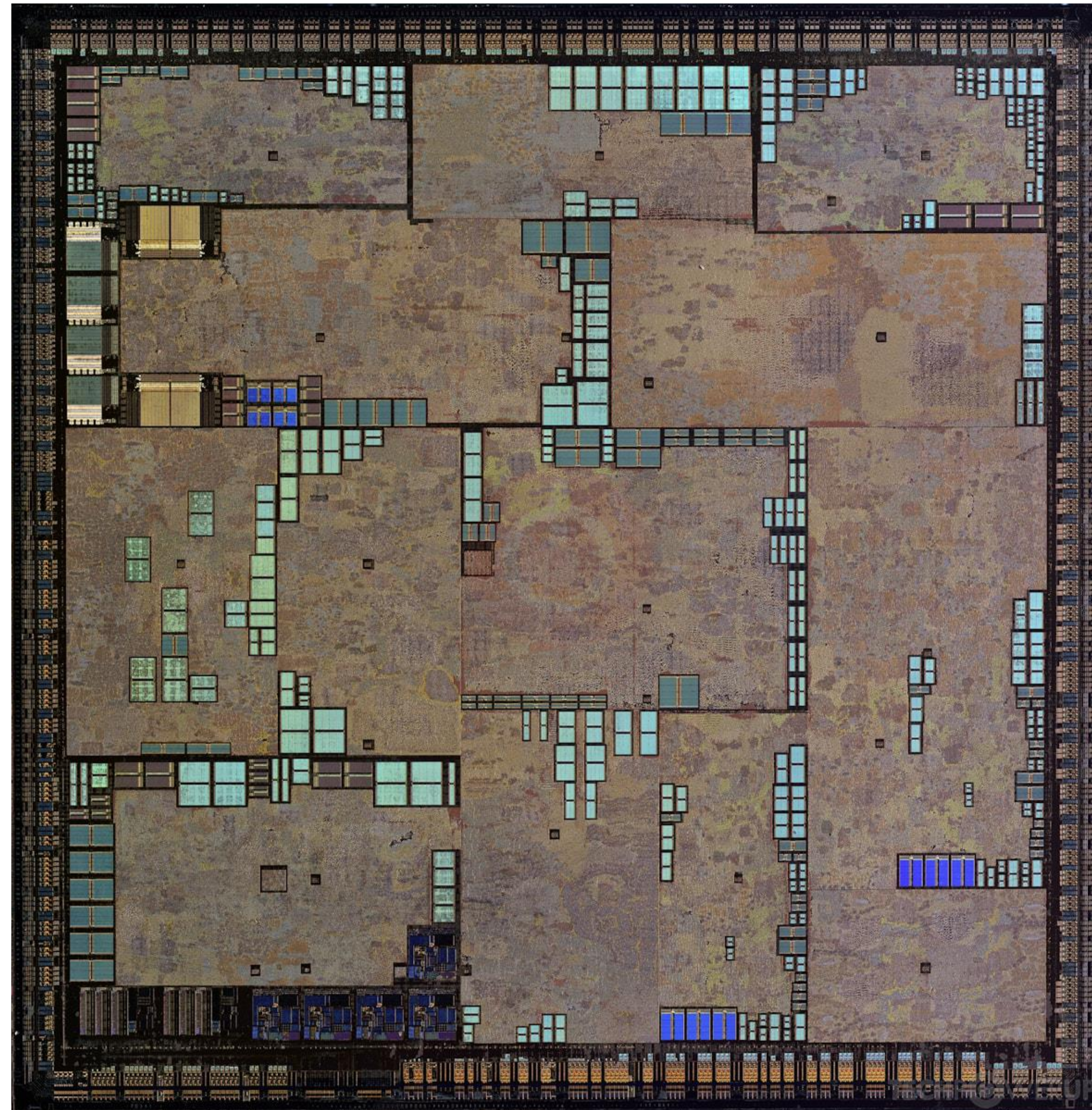


Scalable Physical Design Optimization with GPU Acceleration and Generative AI

Mark Ren

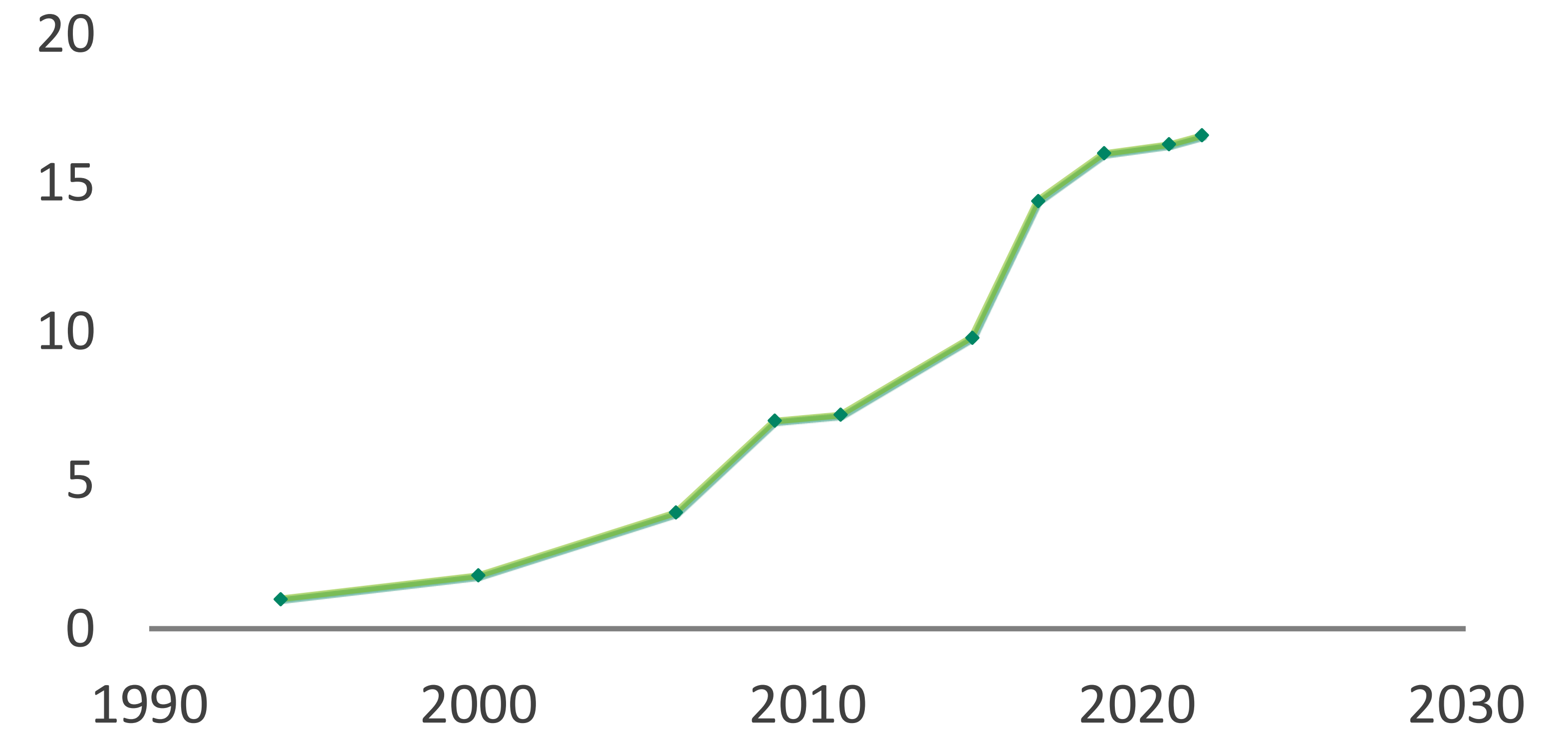
Physical Design Scalability Challenge

Mainly because of P&R tools scalability



NV28 Floorplan (2002)
63M transistors

RELATIVE PHYSICAL DESIGN BLOCK SIZE

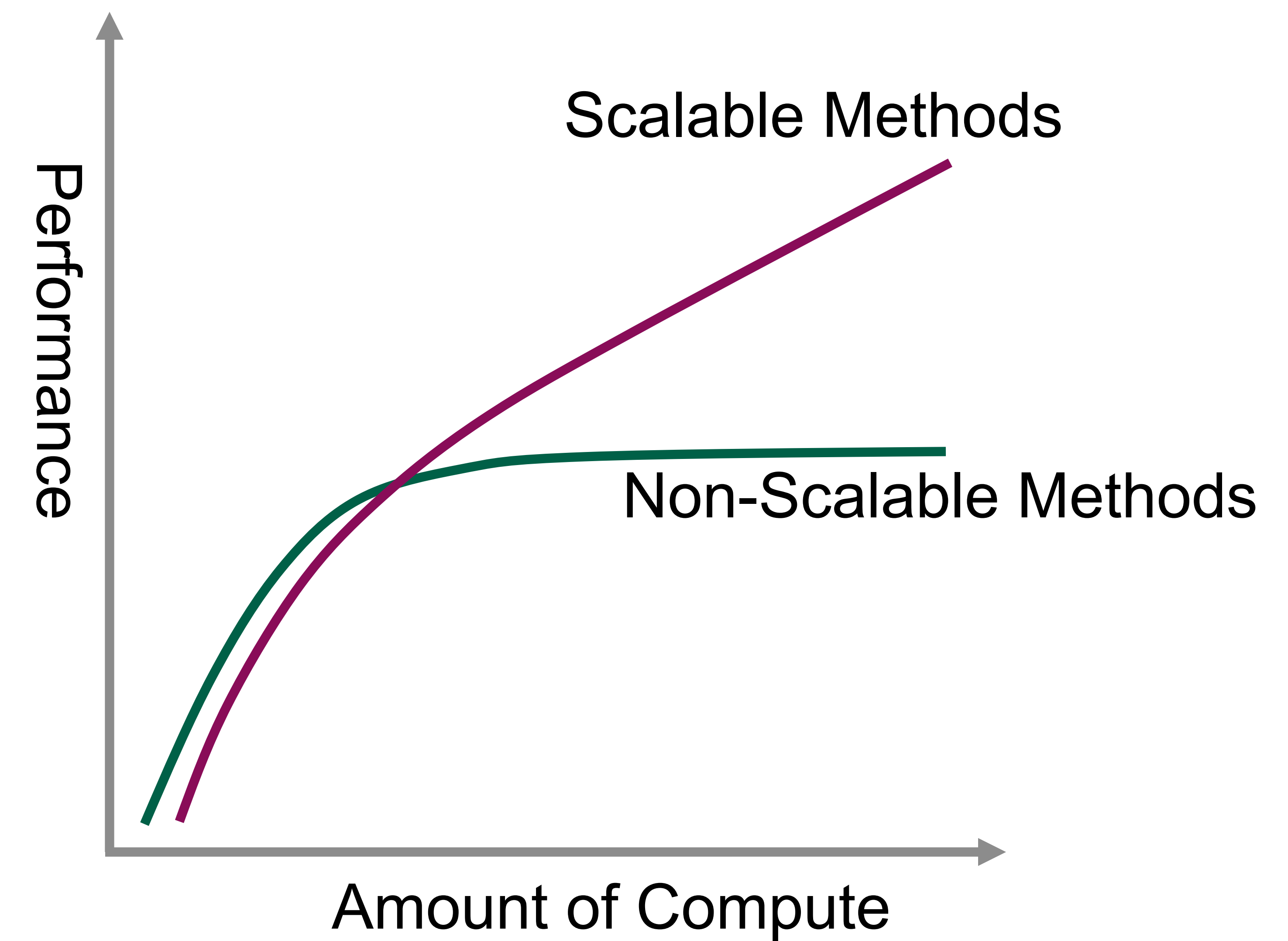
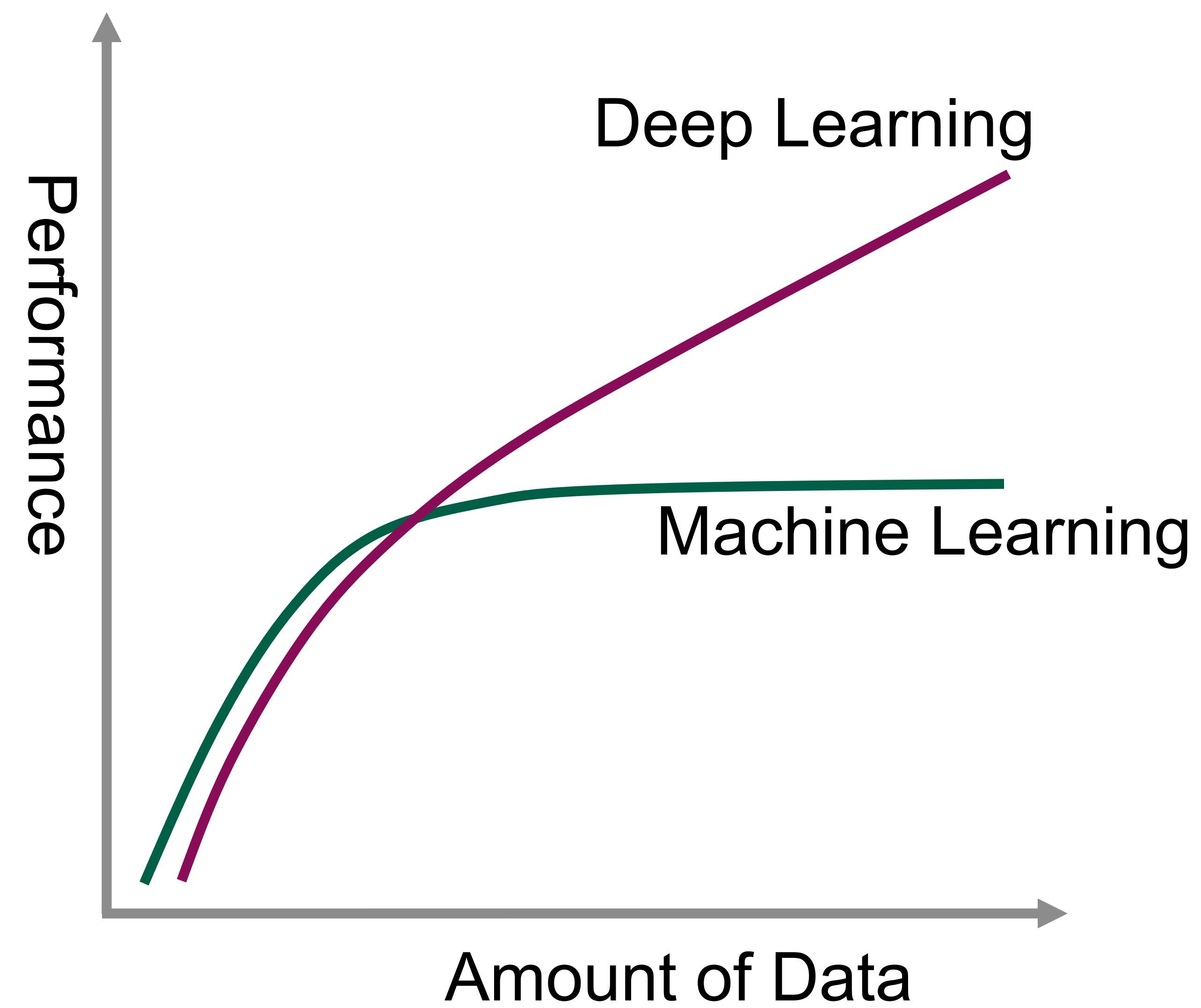


Average physical design block size levels off
>1 week turn around time for 2 Million cell block

Sameer Halepete (VP VLSI @ NVIDIA) Accelerating the Design and Performance of Next Generation Computing Systems with GPUs, ISPD'22

Scalable methods vs non-scalable methods

Key is whether the performance can scale with the compute available



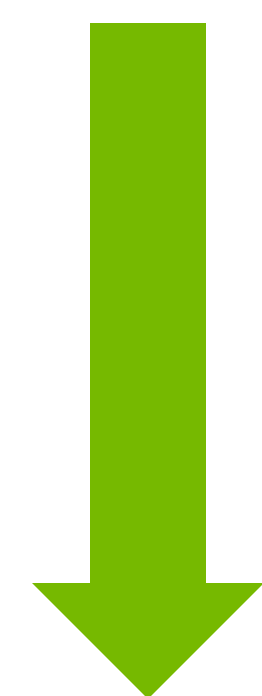
Scalable Method 1: Differentiable Optimization

Leverage DL Frameworks Optimized for GPU

Minimize wirelength and density overflow

$$\min f = WL(x, y) + \mu \sum_{bin} Overflow_{bin}(x, y)^2$$

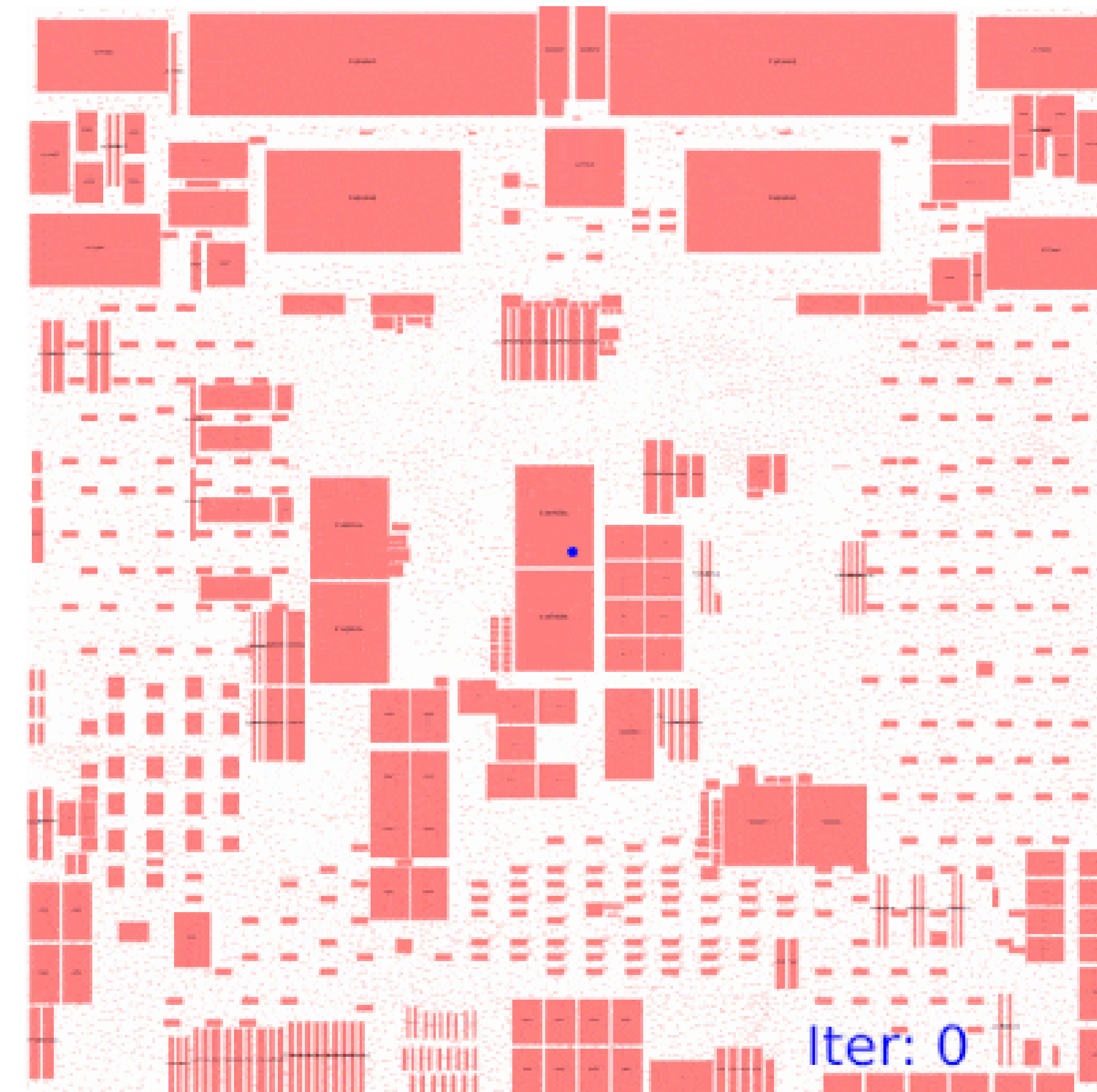
Gradient-based optimization :



$$x = x - \gamma \frac{\partial}{\partial x} f$$
$$y = y - \gamma \frac{\partial}{\partial y} f$$

Neural networks back propagation

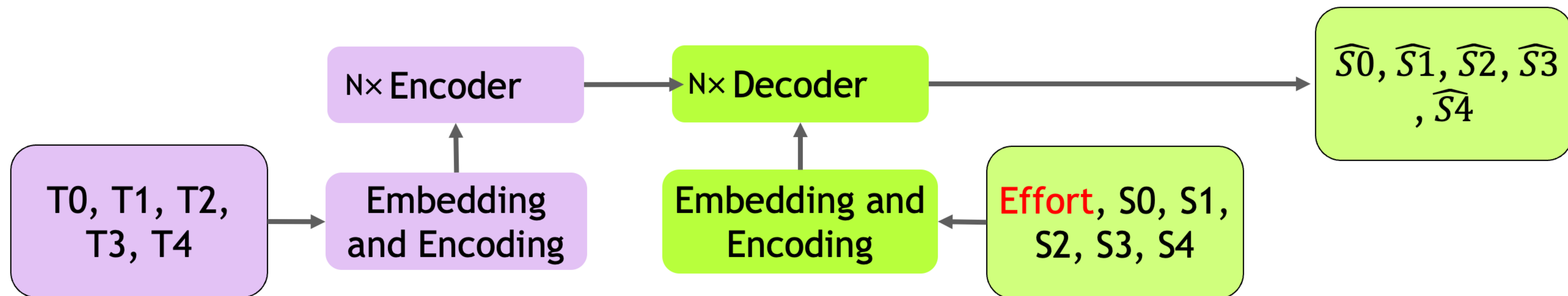
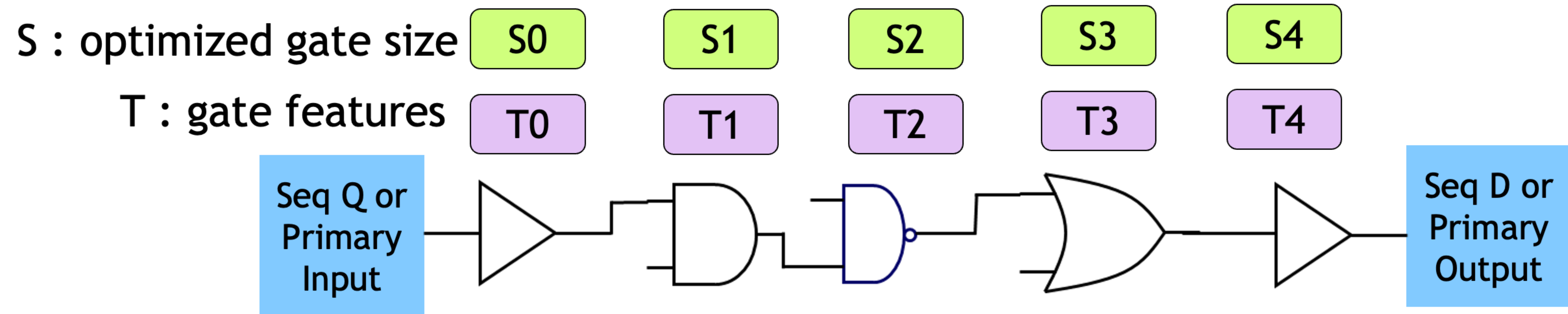
$$\min \sum_i loss(w, x_i)^2 + L_2(w)$$



Accelerate with DL Frameworks on GPU

Scalable Method 2: Generative model trained with SFT

Leverage NN acceleration on GPU and previously optimized data



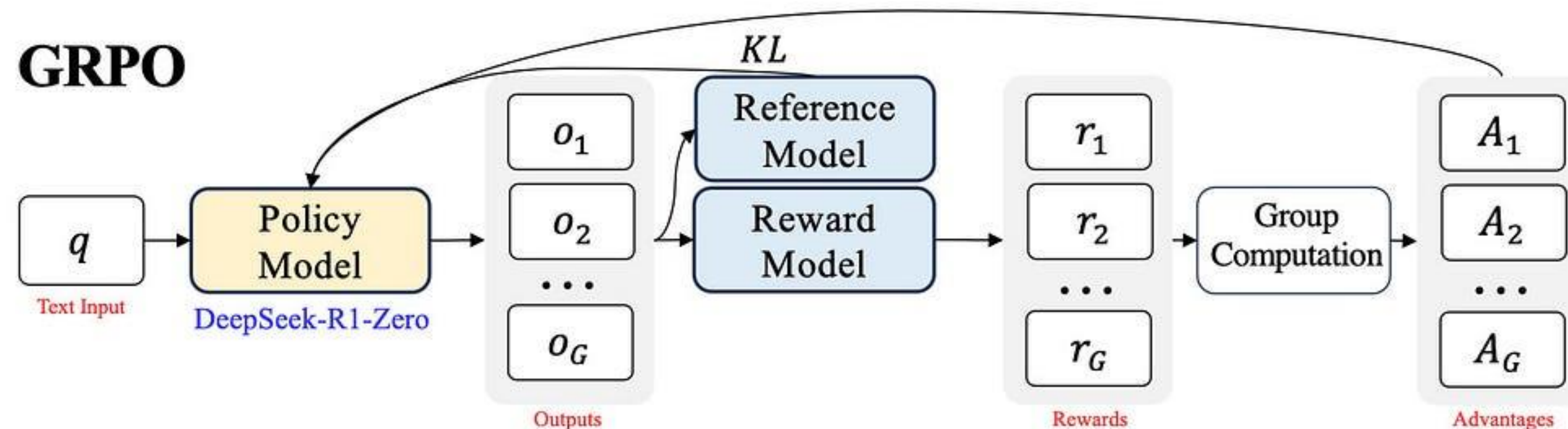
Transsizer: : A Novel Transformer-Based Fast Gate Sizer, ICCAD'22

Scalable Method 3: Generative model trained with RL

What if we do not have enough optimized data? Leverage RL to sample data

Generative model makes RL for optimization scalable

Just replace Policy Model with a Generative Model, and Reward Model with PPA estimation



DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024

How to compute the loss function for optimization?

Static Timing Analysis is the Key for Scalable Optimization

Timing closure is the first-priority optimization objective

Timer is a central component of any physical design system

Differentiable optimization needs a **differentiable timing loss function**

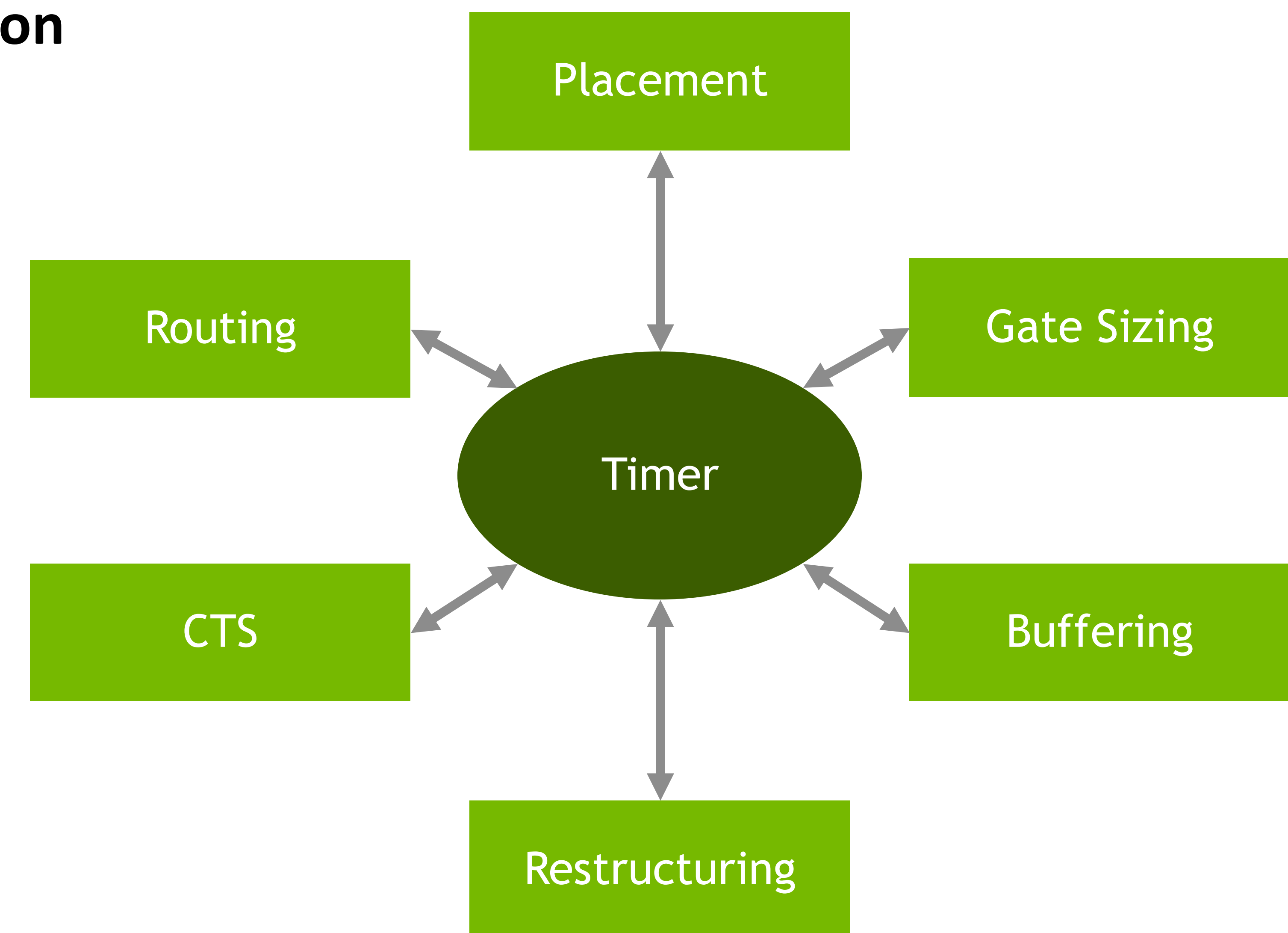
GenAI optimization also needs a **reward function** if using RL

Conventional STA won't work for scalable optimization

Too slow for large scale frequent optimization

Can not calculate the gradient

Need A GPU-accelerated, differentiable STA engine



INSTA: GPU-Accelerated Differentiable Timer

Provide gradient and reward for scalable optimization



Yi-Chen Lu

Accelerate the delay propagation part of STA on GPU

Automatic gradient back with DL framework

Leverage Log-Sum-Exponential (LSE) operator to smooth max operator

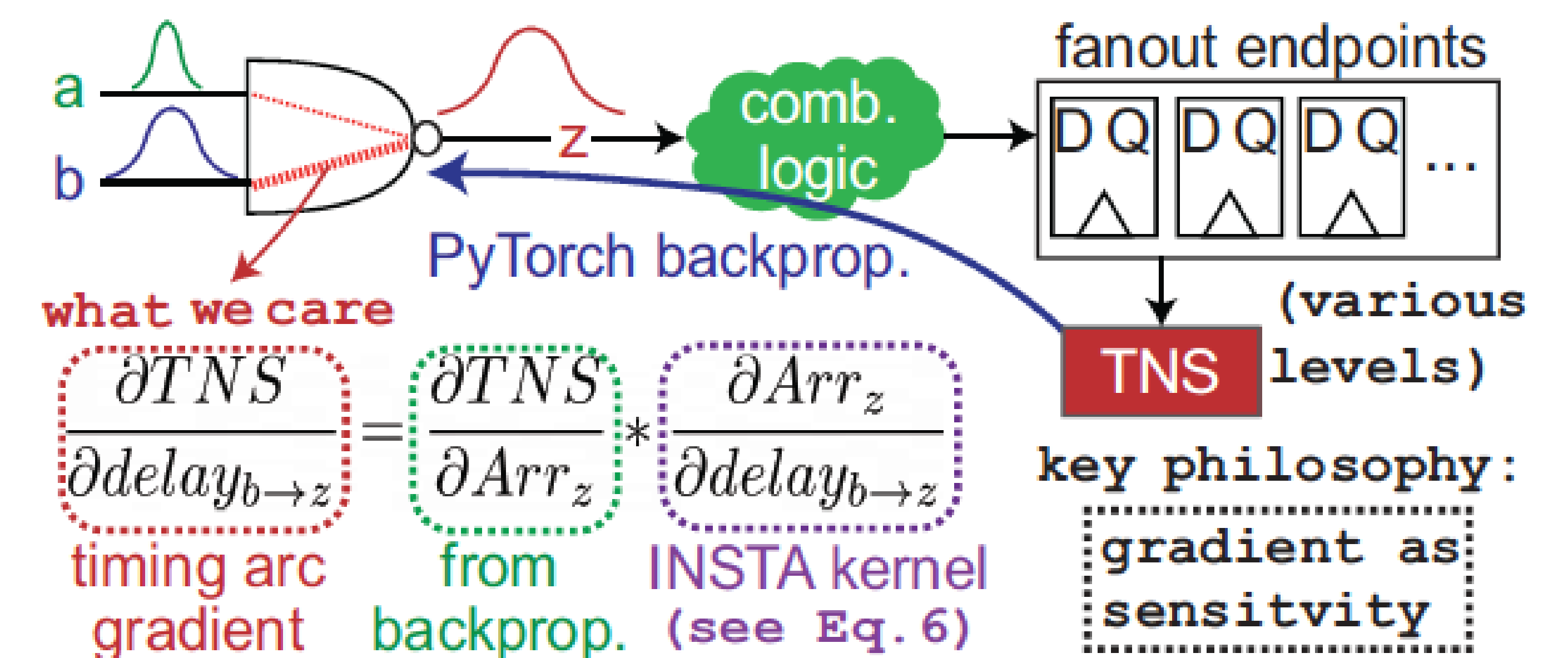
Provide “**timing gradient**” for differentiable optimizations

Provide “**fast reward**” for RL-based optimization

Provide “**sensitivity analysis**” for optimization instance selection

Support statistical propagation and CPPR

0.1 second on 4M cell/15M pin block with 0.999 correlation



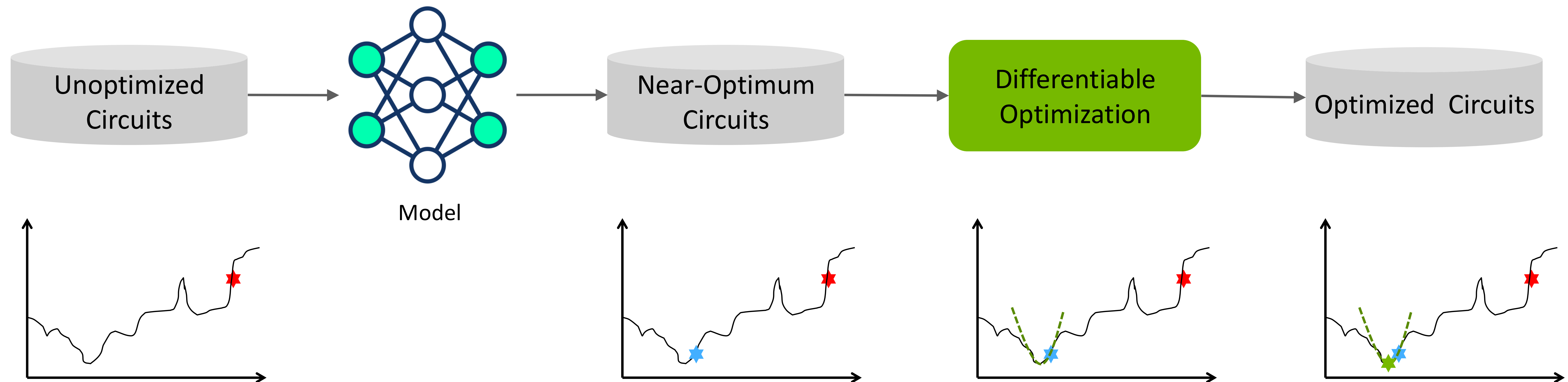
*INSTA: An Ultra-Fast, Differentiable, Statistical Static Timing
Analysis Engine for Industrial Physical Design Applications, DAC'25 BPA*

<https://github.com/NVlabs/INSTA>

Combine Gen AI and Differentiable Optimization

Leverage Gen AI model to generate a solution that is close to the global optimum

Build gradient-based optimization in the global optimum neighborhood to search for the final solution



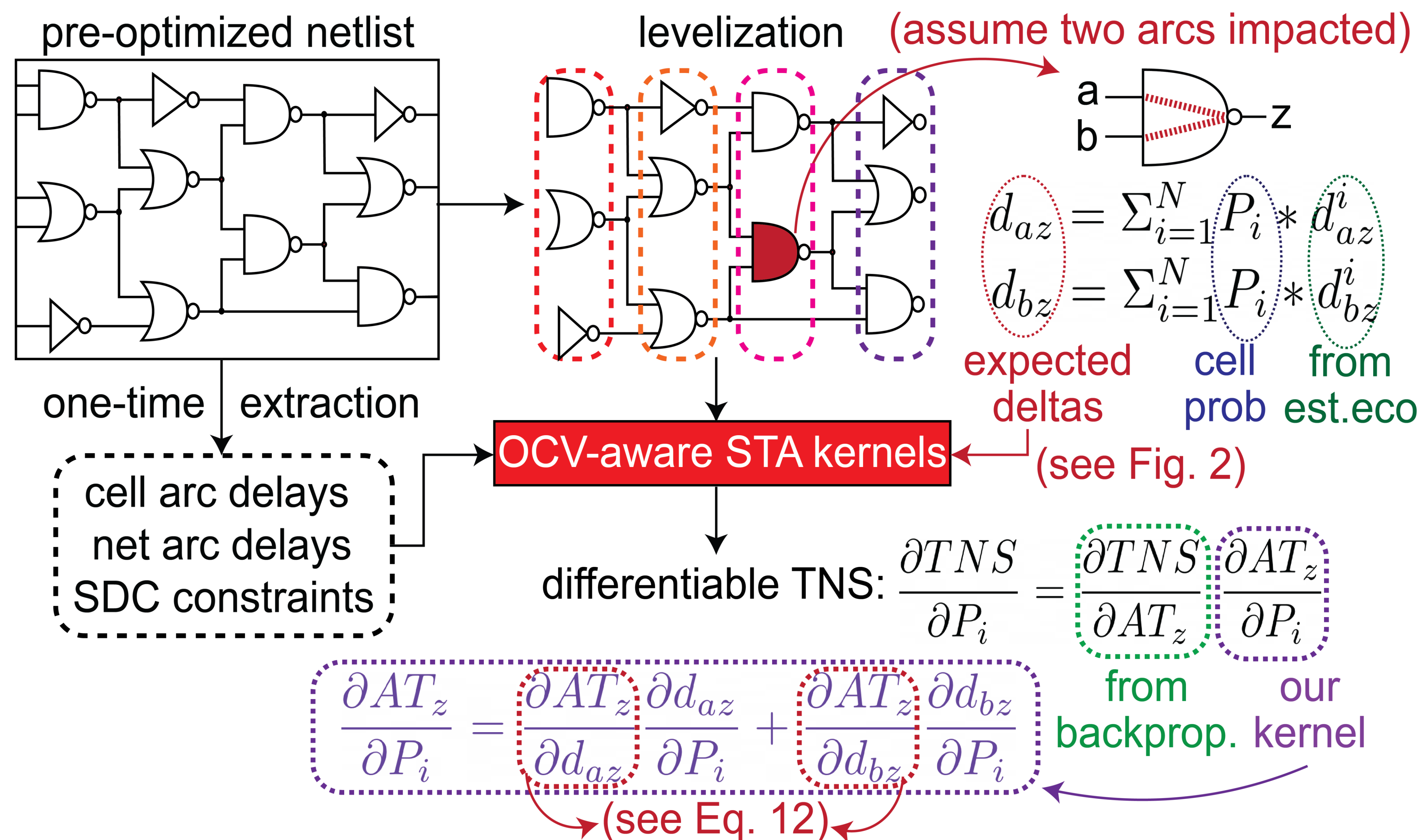


Differentiable Gate Sizing Process

Relax the discrete LibCell optimization with softmax relaxed probability optimization

The INSTA engine is leveraged to perform differentiable timing propagation

Overcome path-based ambiguity by graph-based refinement



LEGO-Sizer Results

Differentiable optimization further improves model-only results

Achieve an average improvement of 26% in TNS and 15.9% in NVE, along with a 113X speedup over commercial baseline on 3nm industrial benchmarks

	Total	reg->reg	in->reg	reg->out	in->out
WNS	-0.834	-0.054	-0.206	-0.834	0.000
TNS	-1708.678	-10.026	-6.283	-1692.369	0.000
NUM	7393	1539	83	5771	0

initial

	Total	reg->reg	in->reg	reg->out	in->out
WNS	-0.834	-0.032	-0.204	-0.834	0.000
TNS	-1703.861	-5.374	-6.118	-1692.369	0.000
NUM	6758	906	81	5771	0

ECO Tool-optimized

Setup violations

	Total	reg->reg	in->reg	reg->out	in->out
WNS	-0.834	-0.035	-0.206	-0.834	0.000
TNS	-1705.311	-6.662	-6.280	-1692.369	0.000
NUM	7230	1376	83	5771	0

Model-only “predicted” @ F1 0.87

Setup violations

	Total	reg->reg	in->reg	reg->out	in->out
WNS	-0.834	-0.033	-0.205	-0.834	0.000
TNS	-1702.863	-4.318	-6.181	-1692.369	0.000
NUM	6666	812	83	5771	0

LEGO: Model + Differentiable Optimization

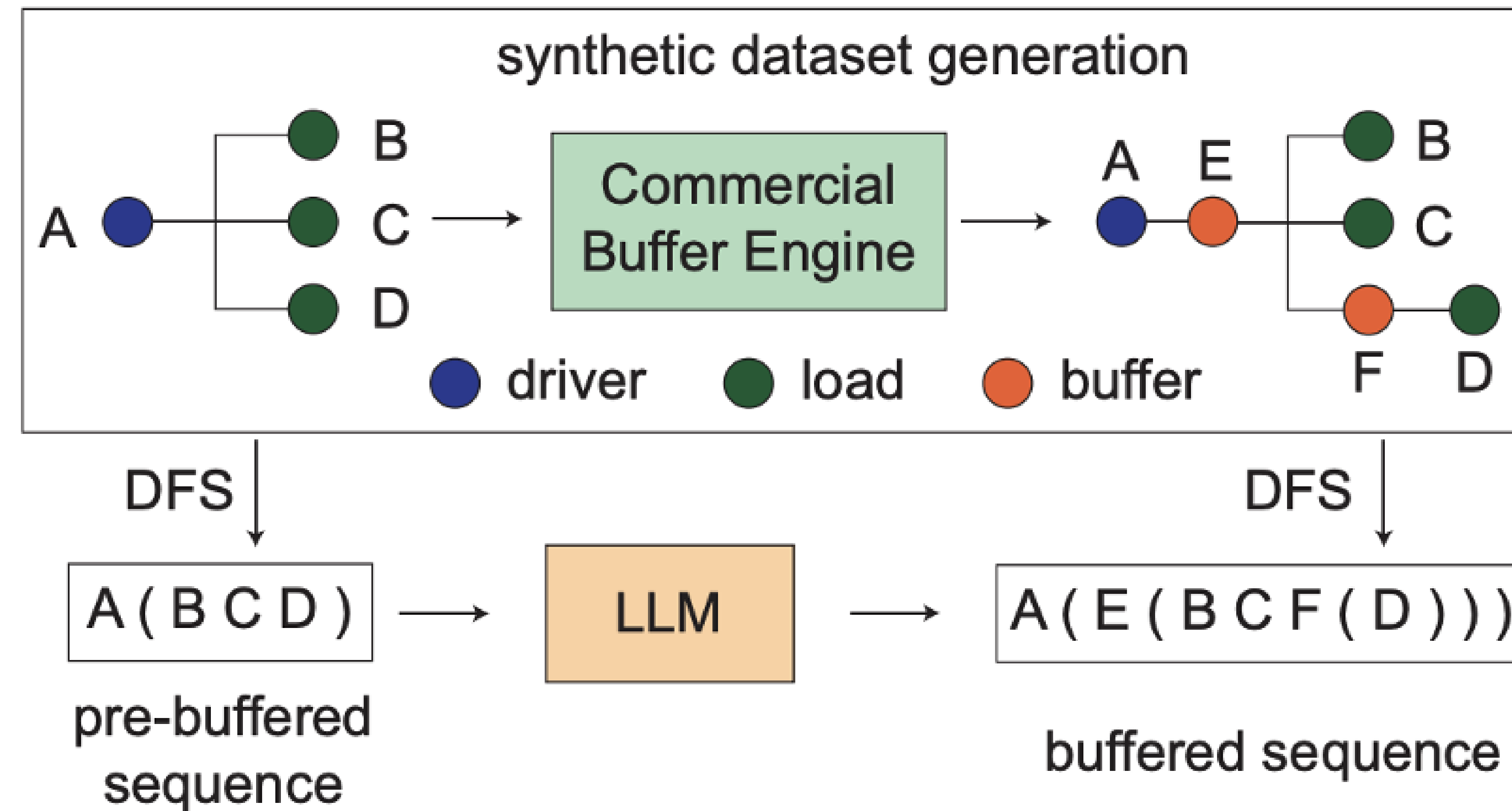
Generative Model for Buffering



Hao-Hsiang Hisao

Model buffering insertion as a DFS sequence

Generate buffer topology, size and location



BUFFALO: PPA-Configurable, LLM-based Buffer Tree Generation via Group Relative Policy Optimization, accepted by ICCAD'25

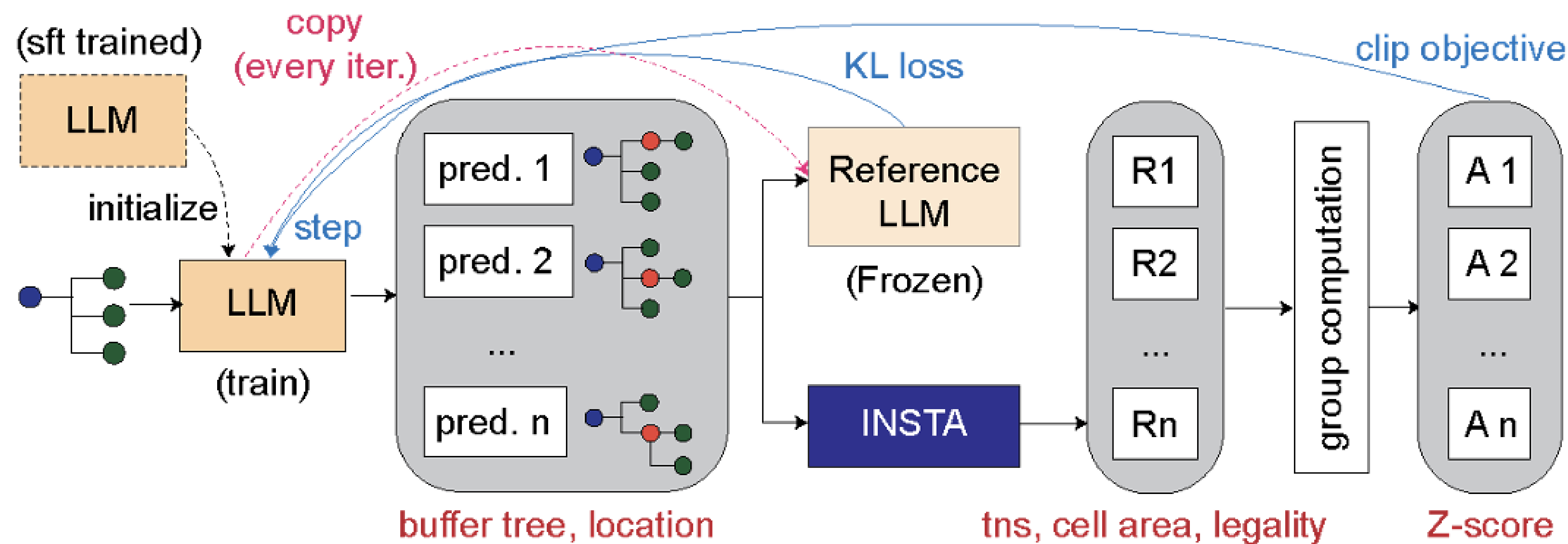
Generative Buffering Model with Reinforcement Learning

GRPO further improves SFT PPA

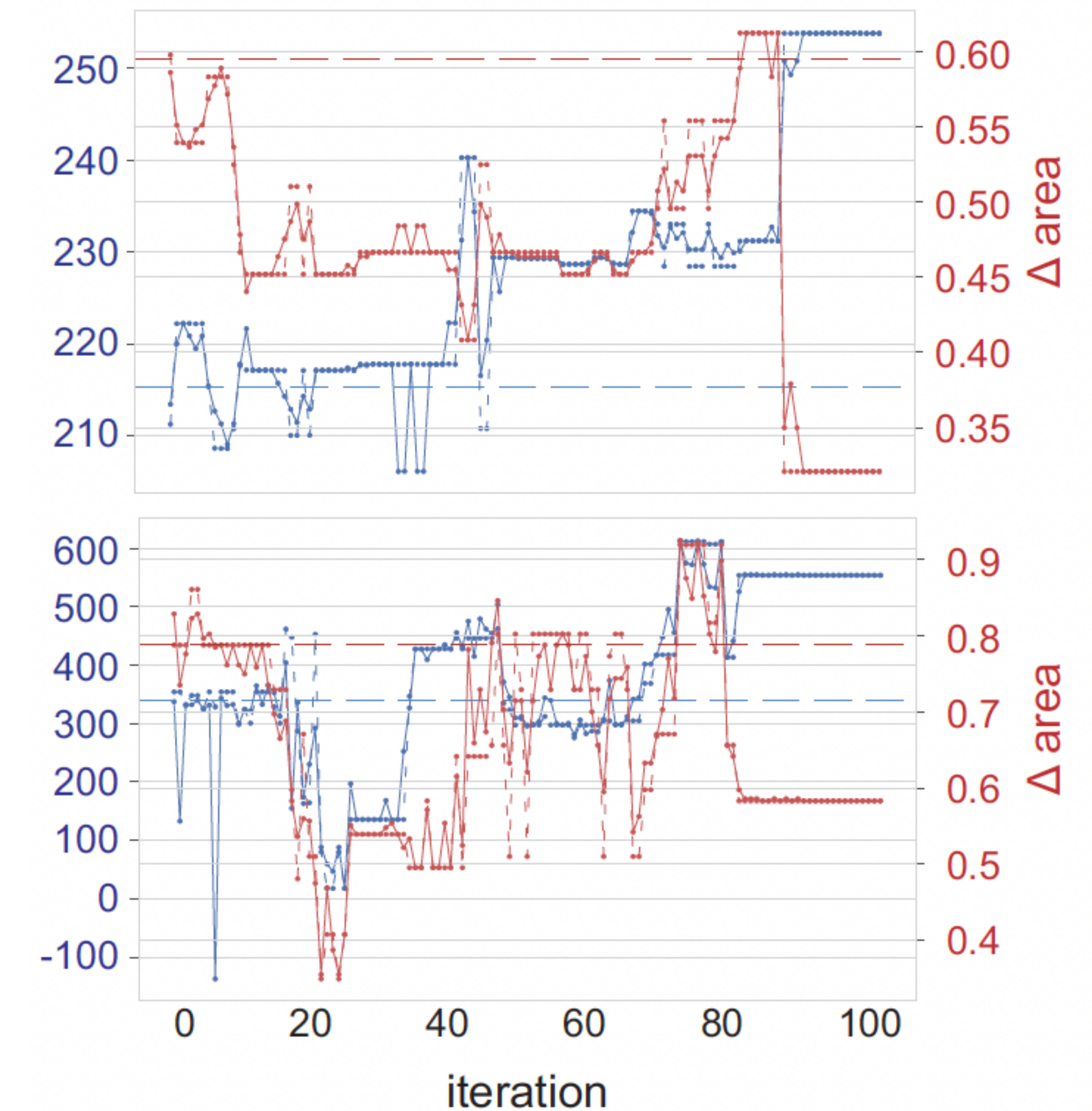
SFT model not enough, train RL (GRPO) with INSTA reward

Two-level RL training: pretraining for each net and deployment at full-chip level with INSTA guided net selection

83x speedup per net, up to 60-70% PPA improvement over the commercial baseline at the end of the flow on ASAP7 benchmarks



GRPO for buffering



PPA Improvement during GRPO

Conclusions

Make physical design scalable with GPU compute power

- Differentiable optimization

- Generative model

- Reinforcement learning

A differentiable GPU-accelerated timing engine is the key enabling tool

Combine these methods together to build scalable and grounded optimization systems



Thanks